

Industrial Internship Report

on

“Student Management System”

Prepared By
Aditya Srivastava

College: Ajay Kumar Garg Engineering College (AKGEC), Ghaziabad

Course: B.Tech — Information Technology (First Year, Second Semester)

Internship Domain: Python Programming

Organization: Upskill Campus, in association with UniConverge Technologies Pvt. Ltd. (UCT)

Duration: 18 June 2026 – 15 July 2026

Submission Date: 16 July 2026

GitHub Repository: github.com/adityasri0512/upskillcampus

LinkedIn: linkedin.com/in/aditya-srivastava-24384a365

Executive Summary

This report presents the details of the Industrial Internship undertaken under Upskill Campus (USC), in association with UniConverge Technologies Pvt. Ltd. (UCT), during the period 18 June 2026 to 15 July 2026. The internship was carried out in the Python programming domain and required the design, development, and documentation of a complete project within the given timeframe.

The project undertaken was a Student Management System — a menu-driven, console-based application built in Python that allows a user to Add, View, Search, Update, and Delete student records. All records are persisted in a structured JSON file (students.json), which acts as a lightweight, file-based database that survives across program runs. The application follows a modular design, with dedicated functions for each operation and shared helper functions to load and save data, wrapped inside a continuously running menu loop until the user chooses to exit.

Over the course of the internship, the project evolved from a basic script into a fully validated CRUD (Create, Read, Update, Delete) application. Particular attention was given to input validation and error handling — for instance, gracefully reporting “Student not found” when an invalid Roll Number is searched, updated, or deleted, rather than letting the program crash. The complete source code, along with a descriptive README.md, was version-controlled and published on GitHub, making the project easy to review, share, and extend.

This internship offered valuable exposure to real-world software development practices — writing clean, modular Python code, working with file handling and JSON serialization, debugging logical and runtime errors, and using Git/GitHub for version control and professional documentation. Beyond the technical skills, it also strengthened problem-solving ability and the discipline of documenting one's own work clearly, which is an essential skill for any engineer.

Overall, this has been a rewarding and practical learning experience. It bridged the gap between classroom concepts of Python and file handling and an actual, working, shareable software project, and has laid a strong foundation for future, more advanced work such as GUI-based or database-backed applications.

Table of Contents

Executive Summary.....	2
1. Preface.....	4
2. Introduction.....	5
2.1 About UniConverge Technologies Pvt. Ltd.....	5
2.2 About Upskill Campus (USC).....	5
3. Objectives.....	7
4. Glossary / Abbreviations.....	8
5. Problem Statement.....	9
6. Existing vs. Proposed System.....	10
7. Proposed Design / Model.....	11
7.1 High-Level Architecture.....	11
7.2 Low-Level Design.....	11
7.3 Flowchart.....	12
8. Technologies Used.....	13
9. Project Modules.....	14
10. Project Structure.....	15
11. GitHub Repository.....	16
12. Add Student Module.....	18
13. View & Search Student Module.....	19
14. Update Student Module.....	20
15. Delete Student Module.....	21
16. students.json — Data File.....	22
17. Testing.....	23
18. Performance Analysis.....	26
19. Learning Outcomes.....	27
20. Future Scope.....	28
21. Conclusion.....	29
22. References.....	30

1 Preface

This four-week Industrial Internship, conducted under Upskill Campus in association with UniConverge Technologies Pvt. Ltd. (UCT), gave me my first structured opportunity to work on an end-to-end software project outside the classroom. As a first-year B.Tech student in Information Technology, this was an important step in bridging the gap between theoretical programming concepts and a real, deliverable piece of software.

Every engineering student eventually needs to move beyond textbook exercises and build something that actually works, is documented properly, and can be shown to someone else. This internship was designed exactly for that purpose — it gave me a well-defined problem statement (build a working Student Management System in Python), a realistic timeline of four weeks, and the responsibility of carrying the project from an empty file to a complete, tested, and version-controlled application.

The programme was planned in a steady, incremental manner: the first week was spent understanding the domain and setting up the development environment (Python and Visual Studio Code); the following two weeks were used to design and implement the core CRUD functionality along with JSON-based storage; and the final week was dedicated to testing, handling invalid inputs gracefully, publishing the project on GitHub, and preparing this report.

Through this process I learned far more than just Python syntax — I learned how to structure a program into functions, how to persist data between runs using files, how to anticipate and handle bad input instead of letting a program crash, and how to write a clear README so that someone else could understand and run my project. Debugging my own logical errors, patiently and methodically, turned out to be one of the most valuable skills this internship reinforced.

I would like to thank Upskill Campus and UniConverge Technologies Pvt. Ltd. for structuring this internship so clearly, and for providing the report template and guidance that made it possible to document this work professionally. I am also grateful to my college, Ajay Kumar Garg Engineering College, for encouraging students to take up such industry-oriented learning opportunities alongside our regular coursework.

To my juniors and peers who will take up similar internships — my only suggestion is to not rush to write code first. Spend time understanding what the program should do, sketch the flow on paper, and only then start implementing. It saves far more time in debugging than it costs in planning.

2 Introduction

Every academic institution, coaching centre, or training programme needs to maintain accurate, up-to-date records of its students — names, ages, courses, and other details that are referred to constantly for administration, communication, and reporting. When such records are kept manually, on paper or in loosely organized spreadsheets, they are slow to search, easy to duplicate or lose, and difficult to keep consistent as the number of students grows.

The Student Management System developed during this internship is a lightweight, console-based Python application that solves this problem at a small scale. It allows a user to Add a new student record, View all existing records, Search for a specific student using their Roll Number, Update an existing record's details, and Delete a record — the five classic CRUD (Create, Read, Update, Delete) operations — through a simple, menu-driven text interface.

Rather than relying on a full database server, the application stores all student data in a single JSON file, `students.json`, using Python's built-in `json` module. This keeps the project self-contained, easy to set up (no installation beyond Python itself), and easy to inspect or back up, since the entire dataset is a single, human-readable text file.

Although modest in scope, this project was deliberately chosen as an internship exercise because it touches almost every fundamental building block of practical programming: control flow and menus, functions and modular design, file handling, structured data (JSON), input validation, and basic software testing — all of which are foundational to larger, more complex systems built later in an engineering career.

2.1 About UniConverge Technologies Pvt. Ltd. (UCT)

UniConverge Technologies Pvt. Ltd. (UCT) is a company established in 2013, working in the Digital Transformation domain and providing industrial solutions with a strong focus on sustainability and Return on Investment (RoI).

To build its products and solutions, UCT leverages a range of cutting-edge technologies, including the Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, and modern front-end frameworks.

UCT's key offerings include the UCT Insight IoT platform for quick deployment of IoT applications with real-time analytics and alerting; the Factory Watch smart-factory platform for production and asset monitoring, OEE, and predictive maintenance; LoRaWAN-based solutions for AgriTech, Smart Cities, and Industrial Monitoring; and Industrial Predictive Maintenance solutions that combine embedded systems, Industrial IoT, and Machine Learning to estimate the remaining useful life of production machinery.

It was UCT, as the industry partner, that structured the problem statement and mentorship for this internship programme, giving participating students an opportunity to experience how a technology-driven company approaches real project execution.

2.2 About Upskill Campus (USC)

Upskill Campus (USC), along with The IoT Academy and in association with UniConverge Technologies, facilitates the smooth execution of the complete internship process — from onboarding and problem-statement allocation to mentorship and final report submission.

USC is a career development platform that delivers personalized, executive-style coaching in a more affordable, scalable, and measurable way, aimed at helping students and early-career professionals build job-ready skills through structured, project-based learning rather than passive lectures.

The IoT Academy, USC's EdTech division, runs long-term executive certification programmes in collaboration with EICT Academies of IIT Kanpur, IIT Roorkee, and IIT Guwahati across multiple technology domains, reflecting the broader ecosystem that this internship is a part of.

3 Objectives

The internship program was designed around the following clear, measurable objectives:

#	Objective
1	Gain hands-on experience of building a real, working Python application from scratch.
2	Understand and apply file handling and JSON-based data persistence in Python.
3	Design and implement a complete CRUD (Create, Read, Update, Delete) system through a menu-driven interface.
4	Apply structured problem-solving and systematic debugging while developing the project.
5	Learn to validate user input and handle invalid or unexpected input gracefully.
6	Use Git and GitHub for version control, code hosting, and professional documentation (README).
7	Improve technical writing and reporting skills by documenting the completed project formally.

4 Glossary / Abbreviations

Term	Meaning
CRUD	Create, Read, Update, Delete — the four basic operations performed on persistent data.
JSON	JavaScript Object Notation — a lightweight, human-readable data-interchange format used here to store student records.
IDE	Integrated Development Environment — Visual Studio Code was used to write, run, and debug this project.
Repository	A storage location on GitHub that holds a project's source code and its complete version history.
README	A documentation file (README.md) that introduces a project, explains its features, and describes how to run it.
UCT	UniConverge Technologies Pvt. Ltd. — the industry partner for this internship.
USC	Upskill Campus — the platform that organized and facilitated this internship.

5 Problem Statement

Small institutions, coaching batches, and student clubs often maintain student information — Roll Number, Name, Age, and Course — using paper registers or loosely structured spreadsheets. This approach has several practical drawbacks:

- Searching for a specific student's record is slow and entirely manual.
- Updating a single field (say, a changed course) means locating and re-writing an entire row by hand.
- There is a high chance of duplicate entries, typos, and lost or damaged records.
- There is no easy way to back up, share, or version the data.

The requirement addressed by this project was to build a lightweight, dependable, console-based system that lets a user Add, View, Search, Update, and Delete student records reliably, with the data persisted safely between runs — without requiring the overhead of installing and configuring a full database server. This makes it especially suitable as a foundational project for learning core CRUD and file-handling concepts, while still being genuinely usable for small-scale record keeping.

6 Existing vs. Proposed System

The table below summarizes the key limitations of the existing, manual approach to student record-keeping and how the proposed Student Management System addresses each of them.

Existing System (Manual / Register-based)	Proposed System (Student Management System)
Records maintained on paper or spreadsheets	Records stored digitally in a structured JSON file (students.json)
Searching a record is slow and fully manual	Instant search by Roll Number through the Search Student option
High chance of human error, duplication, or data loss	Centralized, validated data entry reduces errors and duplicate Roll Numbers
No convenient way to update or delete a specific record	Dedicated Update and Delete operations, each with existence validation
Difficult to back up, share, or track changes to the data	Easily backed up, version-controlled, and shared via GitHub

7 Proposed Design / Model

7.1 High-Level Architecture

At a high level, the system follows a simple, linear flow: the user interacts with the Python console application, which reads from and writes to the JSON file for persistence, and finally displays the result of each operation back to the user in the console.

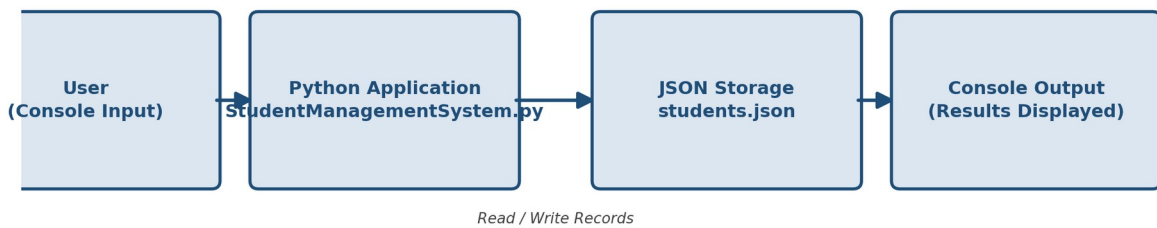


Figure 1: High-Level Architecture of the Student Management System

7.2 Low-Level Design

At the module level, the `main()` function acts as the menu-driven controller. Based on the user's choice, it invokes one of five operation functions — `add_student()`, `view_students()`, `search_student()`, `update_student()`, or `delete_student()`. Each of these, in turn, relies on two shared helper functions: `load_students()`, which reads the current records from `students.json` into memory, and `save_students()`, which writes any changes back to the file.

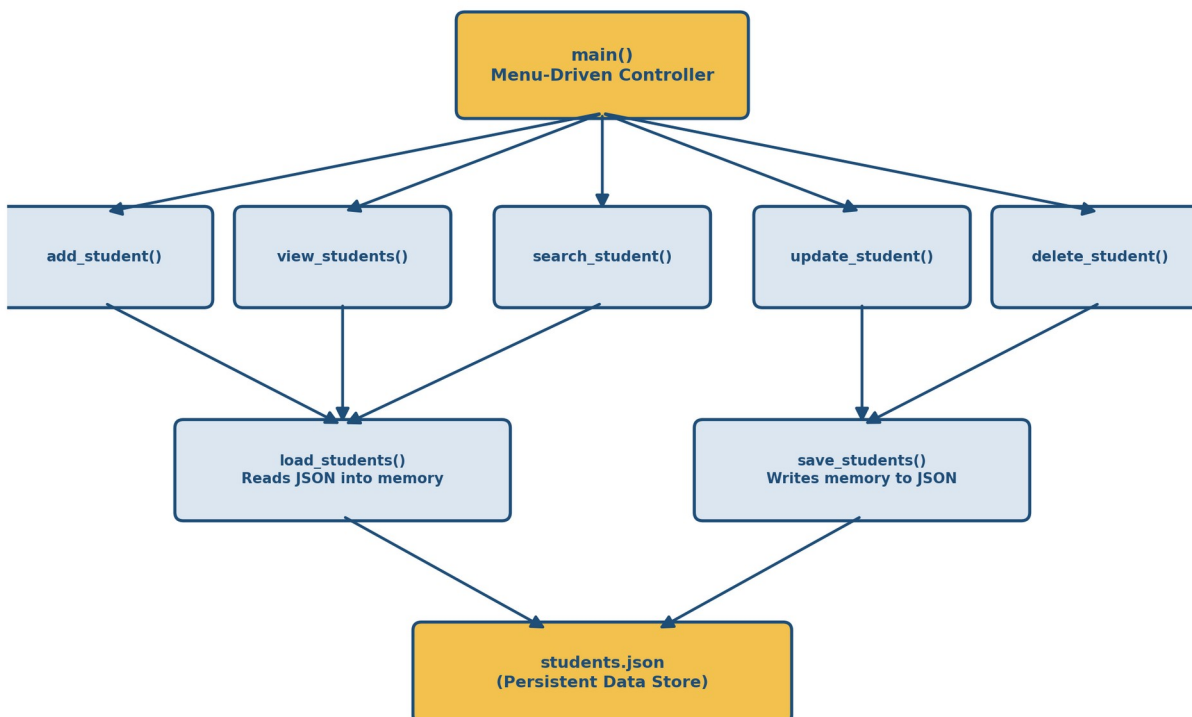


Figure 2: Low-Level Module Design

7.3 Flowchart

The application runs inside a continuous loop: it displays the menu, reads and validates the user's choice, performs the corresponding operation (reading from or writing to students.json as needed), displays the result, and then loops back to the menu — until the user selects the Exit option.

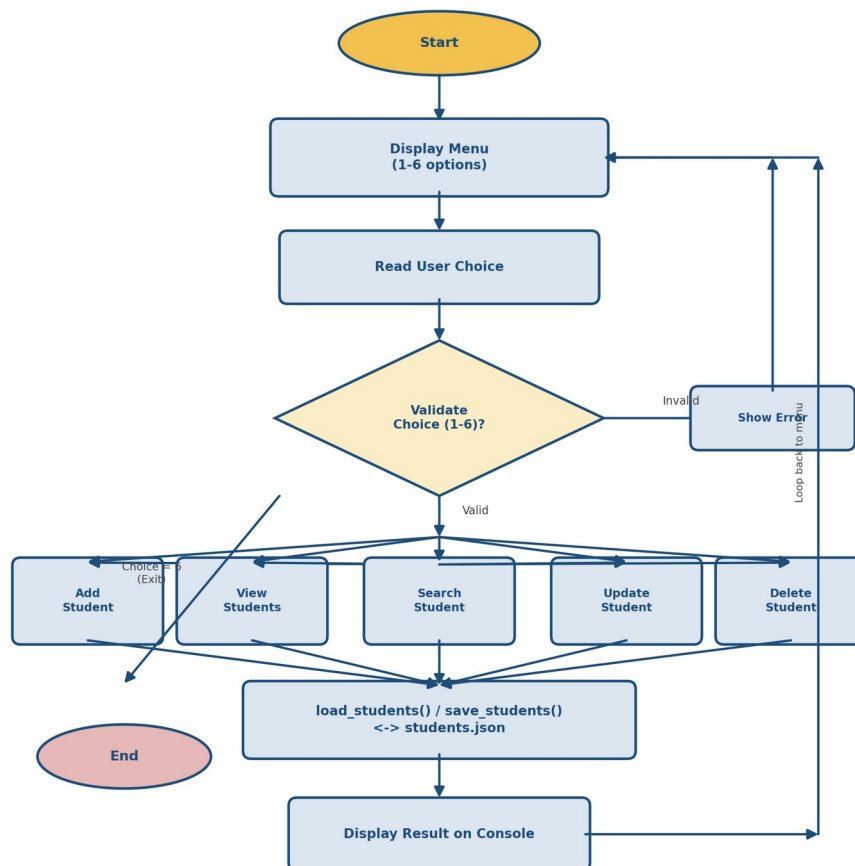


Figure 3: Menu-Driven Application Flowchart

8 Technologies Used

- Python 3.x — the core programming language used for all application logic and control flow.
- JSON — a lightweight, file-based storage format used to persist student records (students.json).
- Visual Studio Code — the IDE used to write, run, and debug the project.
- GitHub — used for version control, code hosting, and publishing the project's documentation.

Software Requirements

- Windows OS
- Python 3.x
- Visual Studio Code
- Git / GitHub account

Hardware Requirements

- Laptop / Desktop computer
- Intel / AMD Processor
- Minimum 4 GB RAM
- 100 MB free storage

9 Project Modules

The application is organized into five user-facing modules, backed by two shared file-handling helper functions:

- Add Student — collects Roll Number, Name, Age, and Course from the user, checks that the Roll Number is not already in use, and appends the new record to students.json.
- View Students — reads and displays every student record currently stored in students.json.
- Search Student — accepts a Roll Number and displays the matching student's details, or reports "Student not found" if no match exists.
- Update Student — accepts a Roll Number, verifies the record exists, and then overwrites its Name, Age, and Course with newly entered values.
- Delete Student — accepts a Roll Number, verifies the record exists, and removes it from students.json.
- load_students() / save_students() — shared helper functions that read from and write to students.json, used internally by all five operations above.

10 Project Structure

The project is organized as a single Visual Studio Code workspace, StudentManagementSystem, containing three files: the main program (StudentManagementSystem.py), the data file (students.json), and the documentation file (README.md). The screenshot below shows the project explorer alongside the editor and the running program in the integrated terminal.

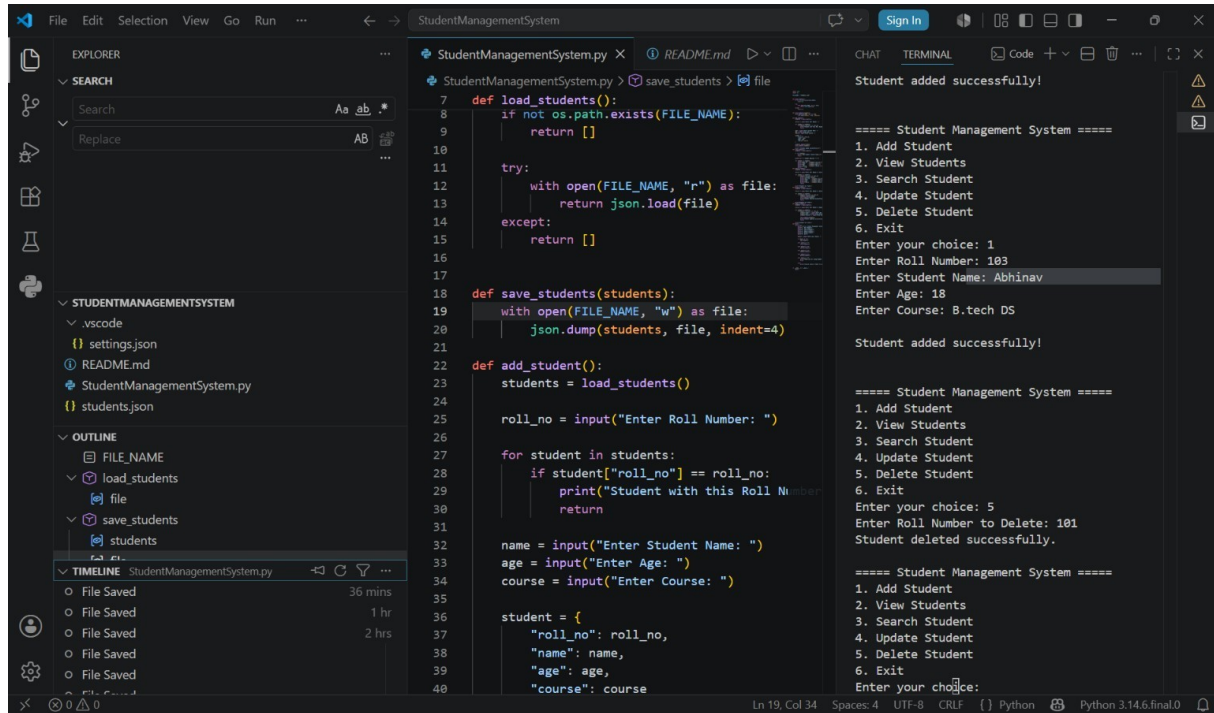


Figure 4: VS Code Project Structure — StudentManagementSystem.py, students.json, and README.md

Keeping the project to just three files — one script, one data file, and one documentation file — keeps it simple to understand, run, and hand over, which was an intentional design choice for a project of this scope.

11 GitHub Repository

The completed project was published to a public GitHub repository at github.com/adityasri0512/upskillcampus, containing the source code, the JSON data file, and a descriptive README.md that documents the project's description, features, technologies used, project structure, and run instructions.

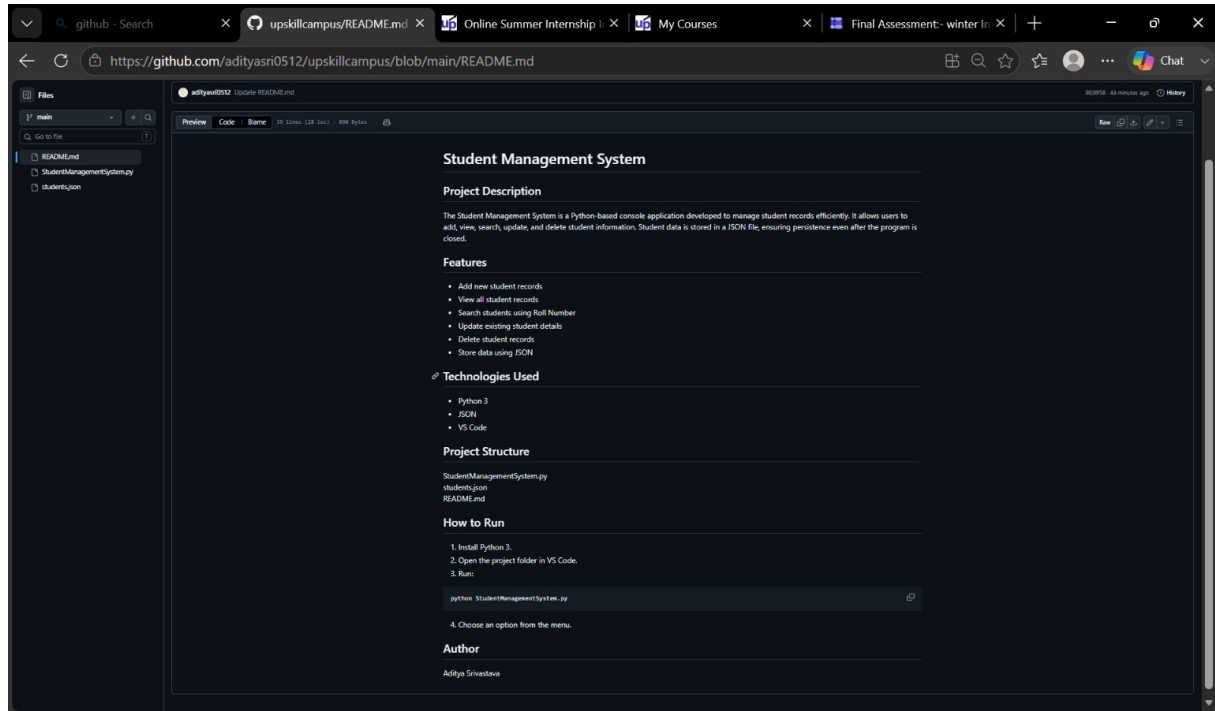


Figure 5: GitHub Repository — README.md View

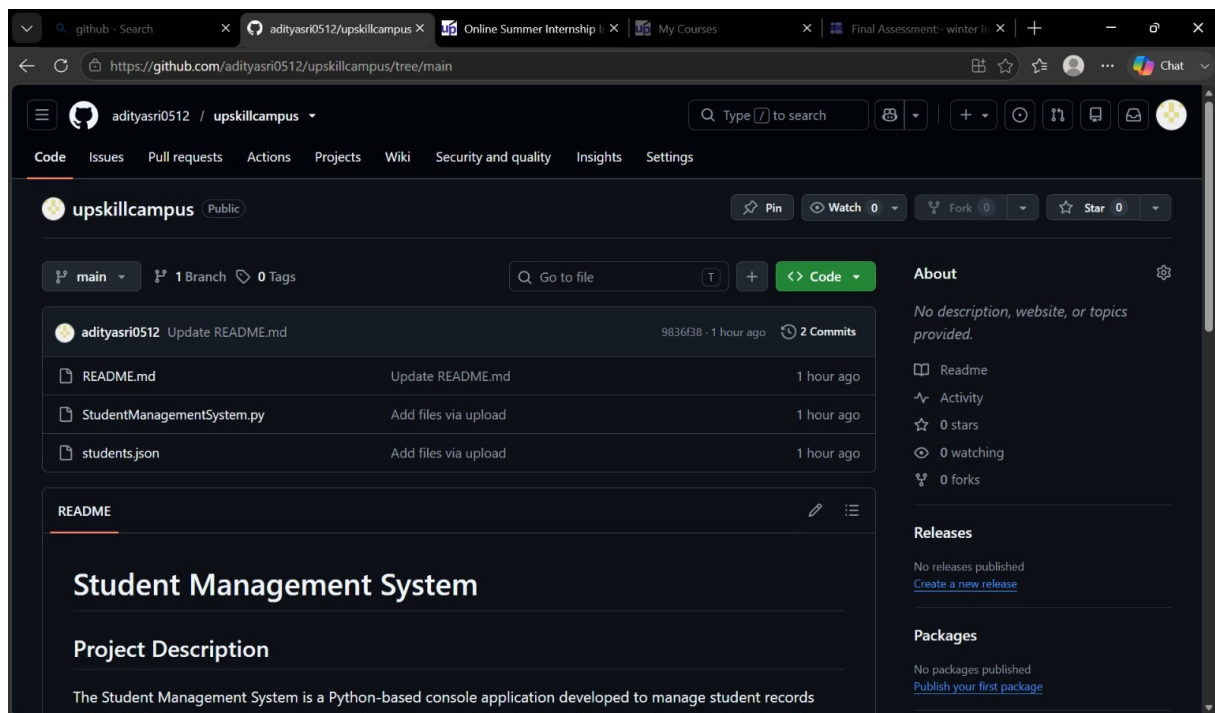
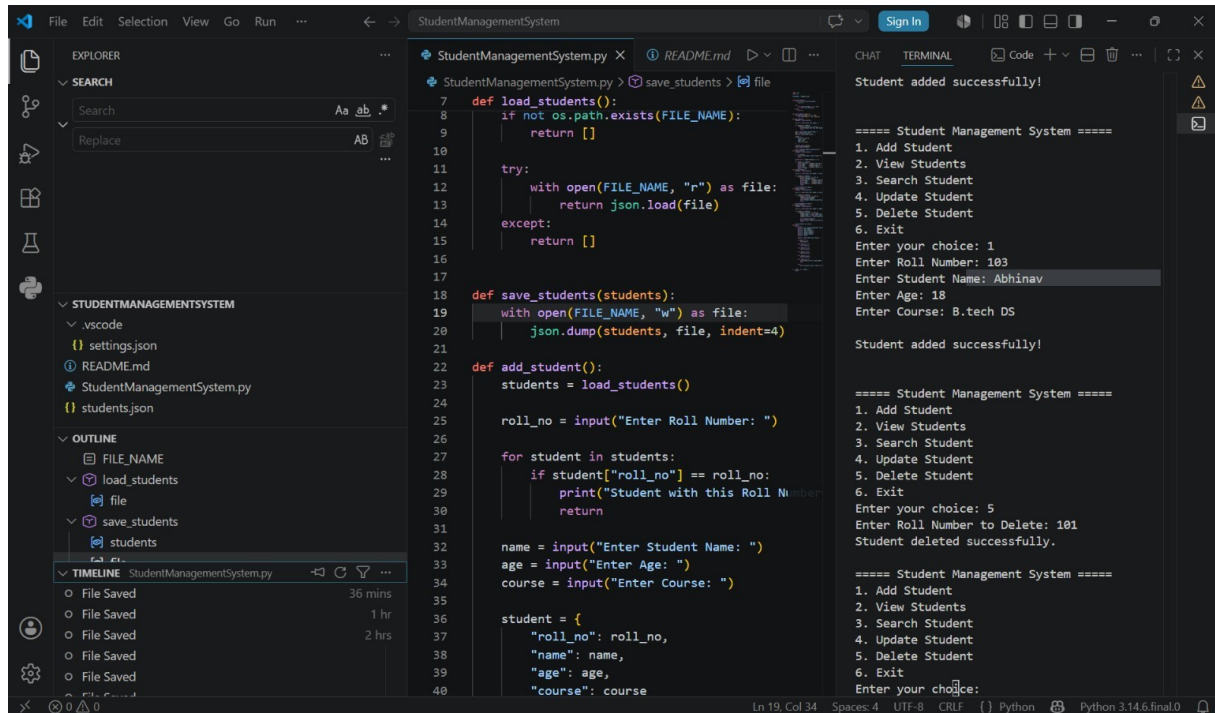


Figure 6: GitHub Repository — Main Page (File Listing)

Publishing the project on GitHub not only provides a permanent, shareable record of the completed internship work, but also gave first-hand practice with Git-based version control, commits, and writing documentation that a stranger could follow to run the project independently.

12 Add Student Module

The Add Student option prompts the user for a Roll Number, Name, Age, and Course. The program first checks that the entered Roll Number does not already exist among the stored records; if it is unique, a new record is appended to students.json and a confirmation message, “Student added successfully!”, is displayed.



The screenshot displays the VS Code interface with the `StudentManagementSystem.py` file open. The file contains the following Python code:

```
7 def load_students():
8     if not os.path.exists(FILE_NAME):
9         return []
10
11     try:
12         with open(FILE_NAME, "r") as file:
13             return json.load(file)
14     except:
15         return []
16
17
18 def save_students(students):
19     with open(FILE_NAME, "w") as file:
20         json.dump(students, file, indent=4)
21
22 def add_student():
23     students = load_students()
24
25     roll_no = input("Enter Roll Number: ")
26
27     for student in students:
28         if student["roll_no"] == roll_no:
29             print("Student with this Roll Number")
30             return
31
32     name = input("Enter Student Name: ")
33     age = input("Enter Age: ")
34     course = input("Enter Course: ")
35
36     student = {
37         "roll_no": roll_no,
38         "name": name,
39         "age": age,
40         "course": course
```

The console output on the right shows the program's execution:

```
Student added successfully!

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 1
Enter Roll Number: 103
Enter Student Name: Abhinav
Enter Age: 18
Enter Course: B.tech DS
Student added successfully!

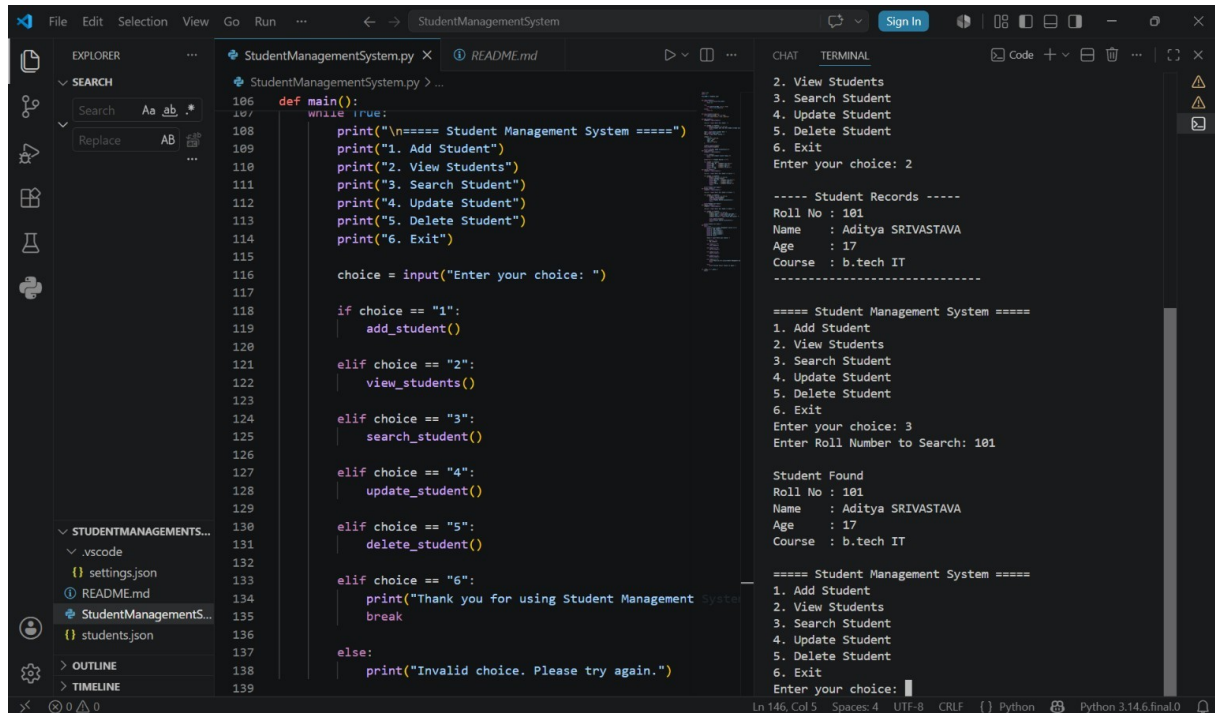
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 5
Enter Roll Number to Delete: 101
Student deleted successfully.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice:
```

Figure 7: Add Student Module — Console Output

13 View & Search Student Module

The View Students option lists every record currently stored in students.json, showing each student's Roll Number, Name, Age, and Course. The Search Student option asks for a specific Roll Number and displays that student's full details when found — as shown below for Roll No. 101.



```
def main():  
    while True:  
        print("\n==== Student Management System =====")  
        print("1. Add Student")  
        print("2. View Students")  
        print("3. Search Student")  
        print("4. Update Student")  
        print("5. Delete Student")  
        print("6. Exit")  
  
        choice = input("Enter your choice: ")  
  
        if choice == "1":  
            add_student()  
        elif choice == "2":  
            view_students()  
        elif choice == "3":  
            search_student()  
        elif choice == "4":  
            update_student()  
        elif choice == "5":  
            delete_student()  
        elif choice == "6":  
            print("Thank you for using Student Management System")  
            break  
        else:  
            print("Invalid choice. Please try again.")
```

```
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice: 2  
  
----- Student Records -----  
Roll No : 101  
Name   : Aditya SRIVASTAVA  
Age    : 17  
Course : b.tech IT  
-----  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice: 3  
Enter Roll Number to Search: 101  
  
Student Found  
Roll No : 101  
Name   : Aditya SRIVASTAVA  
Age    : 17  
Course : b.tech IT  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice:
```

Figure 8: View & Search Student Module — Console Output

14 Update Student Module

The Update Student option asks for the Roll Number of the record to modify, confirms that the record exists, and then prompts for a new Name, Age, and Course to overwrite the existing values. Once saved, the program confirms with “Student updated successfully.”

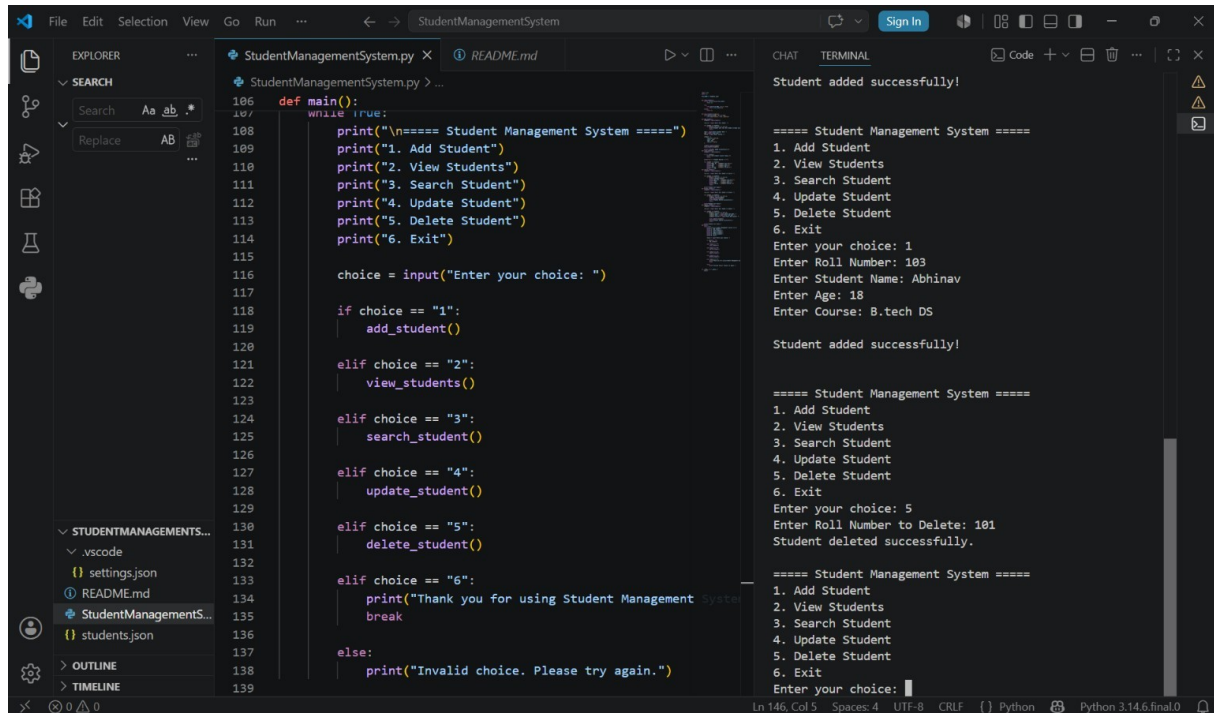
```
def main():  
    while True:  
        print("\n==== Student Management System =====")  
        print("1. Add Student")  
        print("2. View Students")  
        print("3. Search Student")  
        print("4. Update Student")  
        print("5. Delete Student")  
        print("6. Exit")  
  
        choice = input("Enter your choice: ")  
  
        if choice == "1":  
            add_student()  
        elif choice == "2":  
            view_students()  
        elif choice == "3":  
            search_student()  
        elif choice == "4":  
            update_student()  
        elif choice == "5":  
            delete_student()  
        elif choice == "6":  
            print("Thank you for using Student Management System")  
            break  
        else:  
            print("Invalid choice. Please try again.")
```

```
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice: 3  
Enter Roll Number to Search: 101  
  
Student Found  
Roll No : 101  
Name : Aditya SRIVASTAVA  
Age : 17  
Course : b.tech IT  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice: 4  
Enter Roll Number to Update: 101  
Enter New Name: ADITYA  
Enter New Age: 19  
Enter New Course: B.tech CSE  
Student updated successfully.  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice:
```

Figure 9: Update Student Module — Console Output

15 Delete Student Module

The Delete Student option asks for a Roll Number, checks that a matching record exists, removes it from the stored data, and confirms with “Student deleted successfully.” The Exit Program screenshot below shows the graceful shutdown message once the user chooses to leave the application.



```
def main():  
    while True:  
        print("\n==== Student Management System =====")  
        print("1. Add Student")  
        print("2. View Students")  
        print("3. Search Student")  
        print("4. Update Student")  
        print("5. Delete Student")  
        print("6. Exit")  
  
        choice = input("Enter your choice: ")  
  
        if choice == "1":  
            add_student()  
  
        elif choice == "2":  
            view_students()  
  
        elif choice == "3":  
            search_student()  
  
        elif choice == "4":  
            update_student()  
  
        elif choice == "5":  
            delete_student()  
  
        elif choice == "6":  
            print("Thank you for using Student Management System")  
            break  
  
        else:  
            print("Invalid choice. Please try again.")
```

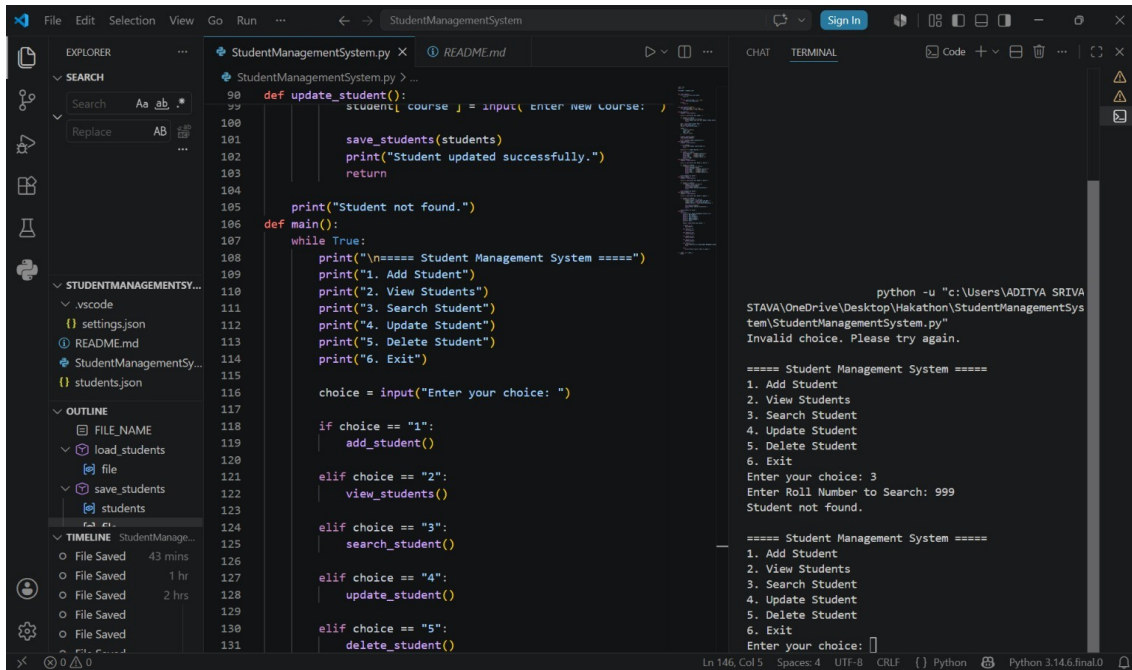
```
Student added successfully!  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice: 1  
Enter Roll Number: 103  
Enter Student Name: Abhinav  
Enter Age: 18  
Enter Course: B.tech DS  
  
Student added successfully!  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice: 5  
Enter Roll Number to Delete: 101  
Student deleted successfully.  
  
==== Student Management System ====  
1. Add Student  
2. View Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Exit  
Enter your choice:
```

Figure 10: Delete Student Module — Console Output

17 Testing

Each module was tested manually with both valid and deliberately invalid input, to confirm that the application behaves predictably and never crashes on bad input. The table below summarizes the test cases executed, including the three invalid-input scenarios captured in the screenshots that follow.

#	Test Case	Expected Outcome	Result
1	Add student with a unique Roll Number	"Student added successfully!"	Pass
2	Add student with a duplicate Roll Number	Program detects the existing Roll Number and does not create a duplicate	Pass
3	View all students	Full list of stored records displayed	Pass
4	Search an existing Roll Number (101)	Matching student's details displayed	Pass
5	Search a non-existent Roll Number (999)	"Student not found."	Pass
6	Update an existing Roll Number (101)	"Student updated successfully."	Pass
7	Update a non-existent Roll Number (111)	"Student not found."	Pass
8	Delete an existing Roll Number (101)	"Student deleted successfully."	Pass
9	Delete a non-existent Roll Number (666)	"Student not found."	Pass
10	Exit the program (choice 6)	Graceful farewell message and clean termination	Pass



```

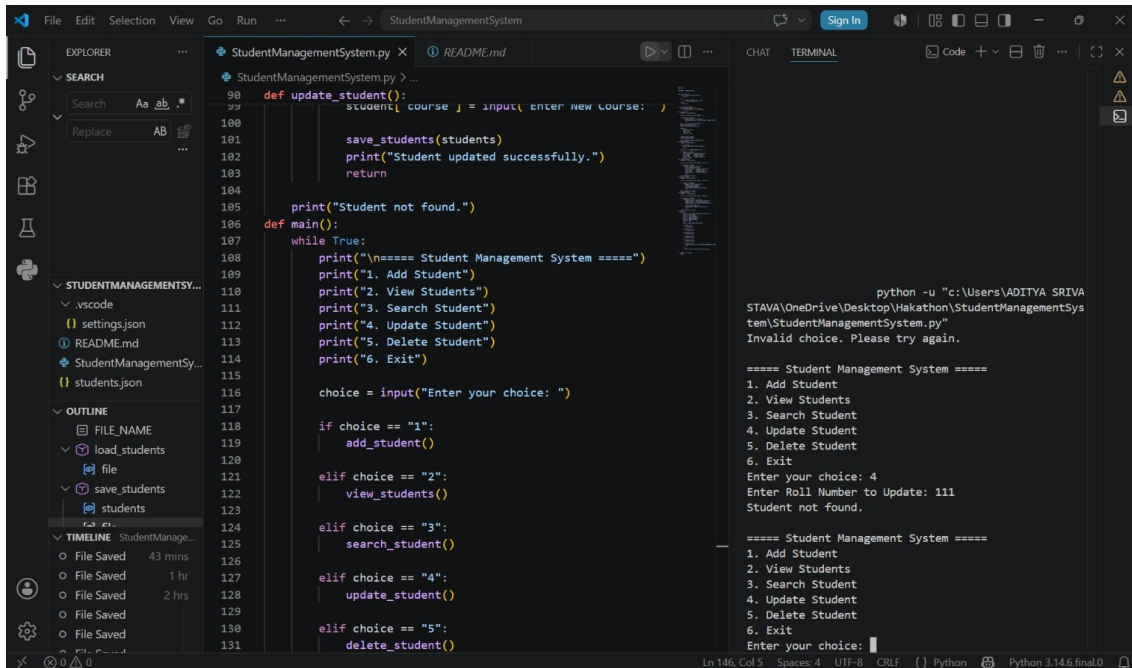
99 def update_student():
100     student[ course ] = input( "Enter new Course: " )
101
102     save_students(students)
103     print("Student updated successfully.")
104     return
105
106     print("Student not found.")
107
108 def main():
109     while True:
110         print("\n===== Student Management System =====")
111         print("1. Add Student")
112         print("2. View Students")
113         print("3. Search Student")
114         print("4. Update Student")
115         print("5. Delete Student")
116         print("6. Exit")
117
118         choice = input("Enter your choice: ")
119
120         if choice == "1":
121             add_student()
122
123         elif choice == "2":
124             view_students()
125
126         elif choice == "3":
127             search_student()
128
129         elif choice == "4":
130             update_student()
131
132         elif choice == "5":
133             delete_student()

```

python -u "c:\Users\ADITYA SRIVA
STA\OneDrive\Desktop\Hakathon\StudentManagementSys
tem\StudentManagementSystem.py"
Invalid choice. Please try again.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 3
Enter Roll Number to Search: 999
Student not found.

Figure 12: Search Student — Invalid Roll Number (999)



```

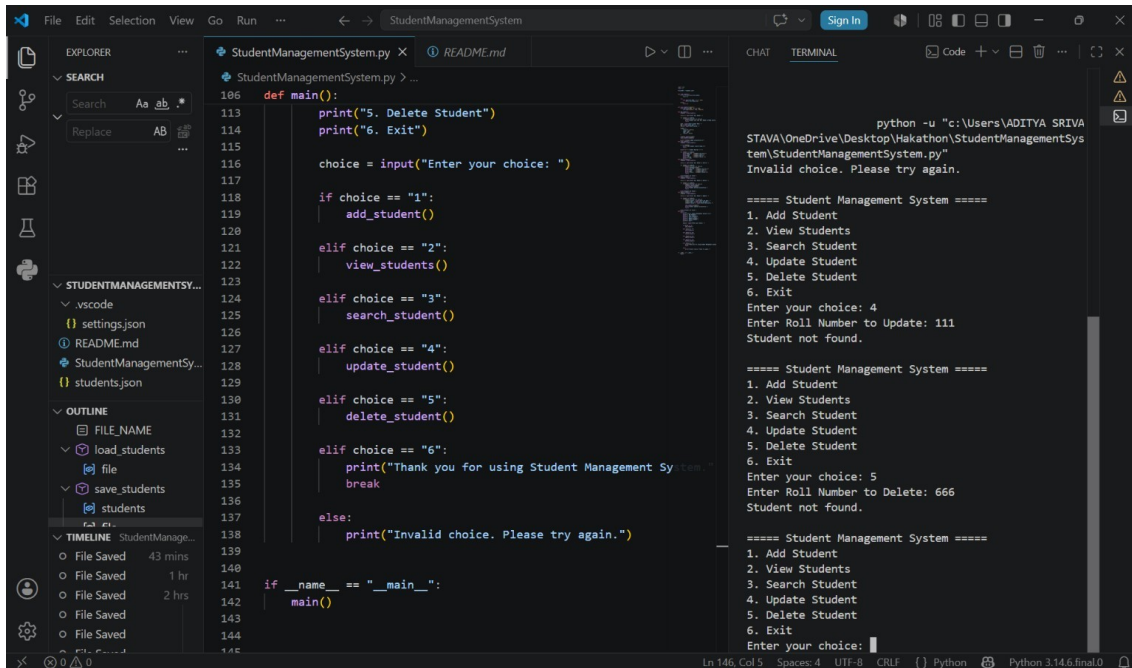
99 def update_student():
100     student[ course ] = input( "Enter new Course: " )
101
102     save_students(students)
103     print("Student updated successfully.")
104     return
105
106     print("Student not found.")
107
108 def main():
109     while True:
110         print("\n===== Student Management System =====")
111         print("1. Add Student")
112         print("2. View Students")
113         print("3. Search Student")
114         print("4. Update Student")
115         print("5. Delete Student")
116         print("6. Exit")
117
118         choice = input("Enter your choice: ")
119
120         if choice == "1":
121             add_student()
122
123         elif choice == "2":
124             view_students()
125
126         elif choice == "3":
127             search_student()
128
129         elif choice == "4":
130             update_student()
131
132         elif choice == "5":
133             delete_student()

```

python -u "c:\Users\ADITYA SRIVA
STA\OneDrive\Desktop\Hakathon\StudentManagementSys
tem\StudentManagementSystem.py"
Invalid choice. Please try again.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 4
Enter Roll Number to Update: 111
Student not found.

Figure 13: Update Student — Invalid Roll Number (111)



```
def main():
    print("5. Delete Student")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        update_student()
    elif choice == "5":
        delete_student()
    elif choice == "6":
        print("Thank you for using Student Management System")
        break
    else:
        print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

```
python -u "c:\Users\ADITYA SRIVASTAVA\Desktop\Hakathon\StudentManagementSystem\StudentManagementSystem.py"
Invalid choice. Please try again.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 4
Enter Roll Number to Update: 111
Student not found.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 5
Enter Roll Number to Delete: 666
Student not found.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 
```

Figure 14: Delete Student — Invalid Roll Number (666)

In every invalid-input case, the program reported “Student not found.” and safely returned to the main menu instead of crashing or leaving the data file in an inconsistent state — confirming that the validation logic works as intended.

18 Performance Analysis

Although the Student Management System is a small-scale console application rather than a production system, it is worth briefly evaluating it against the constraints that matter for real-world software:

- Reliability — All file reads and writes go through the `load_students()` / `save_students()` helper functions, ensuring the JSON file is always written in a consistent, valid format after every operation.
- Speed — Since all records are loaded into memory as a Python list before any operation, Add, View, Search, Update, and Delete are effectively instantaneous for the small-to-moderate record counts this project targets.
- Storage — `students.json` remains only a few kilobytes even with a few hundred records, since JSON is a compact text format with no database engine overhead.
- Limitations — Being file-based, the system is not designed for simultaneous multi-user access, and does not use indexing, so search performance would degrade for very large datasets (tens of thousands of records) — a reasonable trade-off given the project's scope and intended scale.

19 Learning Outcomes

This internship strengthened both technical and professional skills, most notably:

- Python Programming — writing clean, modular, function-based code.
- File Handling — reading from and writing to files using Python.
- JSON — structuring, storing, and parsing data using Python's json module.
- CRUD Operations — implementing Create, Read, Update, and Delete logic correctly and safely.
- Problem Solving — breaking a broad requirement down into small, testable functions.
- Debugging — methodically tracing and fixing logical and runtime errors.
- GitHub — using Git for version control and publishing a project professionally.
- VS Code — using a modern IDE efficiently, including its integrated terminal.
- Documentation — writing a clear README and this formal internship report.

20 Future Scope

While the current version fulfills all the core requirements, several enhancements could extend it into a more complete, production-ready system:

- Graphical User Interface (GUI) — replacing the console menu with a desktop or web interface for easier use.
- MySQL Integration — moving from a JSON file to a relational database for larger datasets and stronger data integrity.
- Login System — adding authentication so only authorized users can modify student records.
- Cloud Database — hosting the data remotely so it can be accessed from multiple devices.
- Export to Excel / PDF — allowing student records to be exported for reporting or printing.
- Student Photos — attaching a photograph to each student record.
- Backup & Restore — adding a dedicated feature to back up and recover students.json safely.

21 Conclusion

This four-week Industrial Internship with Upskill Campus and UniConverge Technologies Pvt. Ltd. provided a genuinely practical, hands-on introduction to real project development — starting from a problem statement and ending with a tested, documented, and publicly hosted Python application. Building the Student Management System required not just writing code, but thinking through data persistence, input validation, and how to present a finished project clearly to others.

Beyond the specific technical skills of Python, JSON, and Git, the internship reinforced habits that matter for any engineer: planning before coding, testing deliberately rather than assuming code works, and documenting a project so that someone else could pick it up and understand it. As a first-year student, this experience has been an encouraging and confidence-building first step into industry-style software development, and I look forward to building on it with more advanced, larger-scale projects.

22 References

- [1] Python Software Foundation, Python 3 Documentation — docs.python.org/3
- [2] GitHub Docs — docs.github.com
- [3] JSON.org, Introducing JSON — json.org
- [4] Visual Studio Code Documentation — code.visualstudio.com/docs
- [5] Upskill Campus — upskillcampus.com
- [6] Project Repository — github.com/adityasri0512/upskillcampus